



AI with Rav

Learn AI the way you actually think.



DAY 7 of 30

OPEN SOURCE CONTRIBUTION

Write the PR Description Like You Value Their Time

WHAT YOU'LL LEARN TODAY

- › How to make a PR reviewable in under a minute
- › Why a table beats a paragraph
- › What 7 merged PRs are worth on a CV
- › What I would do differently



Write the PR Description Like You Value Their Time

In One Line: The last merge. Same kind of PR as yesterday, bigger repo, and one thing done better — the description made the review almost free.

The PR

sentence-transformers, 18 lines added, nothing removed. Five public methods with no docstrings, across two files, in a codebase where everything around them was documented.

By now you know why that qualifies: **missing**, not merely improvable. Same test as Day 6.

The part worth copying

The description did not describe the change in prose. It laid it out in a table:

Method	Added
Dense.forward	Args / Returns
Dense.get_embedding_dimension	Returns
Dense.get_config_dict	Returns
Dense.save	Args
to_scipy_coo	Args / Returns / Example

A reviewer opening this knows the full scope in about five seconds: five methods, two files, docstrings only.



What a maintainer needs, in order

1	SCOPE	what changed, as a list or table	5 seconds
2	WHY	one sentence on the gap it fills	10 seconds
3	RISK	"no logic changes" / what is opt-in	5 seconds
4	PROOF	test that fails without the fix	the review
5	CHECKS	linter / pre-commit already run	0 seconds

A table beats a paragraph: it hands over the list instead of hiding it.

What a maintainer needs from a PR description, in the order they need it.

Why the table works

A paragraph forces the reviewer to *extract* the list. A table hands it over.

More importantly, it lets them check completeness. They can see **Dense** has four methods listed and ask "is that all of them?" — a question they can answer in one glance at the file. Prose hides that question.

Write the description for someone who has thirty seconds, has never seen your name, and has eleven other PRs open. Scope first, reasoning second, everything else after.

Explaining the why in one line

The description also said why the gap mattered:

"The Dense class had a thorough class-level docstring but its individual methods were undocumented, making it harder to understand what each method expects and returns at a glance."

One sentence, and it does real work. It shows this is not a drive-by — I noticed the class docs were good and the method docs were absent, which is exactly the *missing versus improvable* distinction.



The Example that earned its place

For `to_scipy_coo` I added Args, Returns, **and** a short Example. The other four got no example. That was deliberate. `to_scipy_coo` converts to a sparse matrix format where the shape of the output is not obvious from the signature. The **Dense** methods are self-evident once you know the arguments.

Do not add an example to every function to look thorough. An example on an obvious function is noise, and it is one more thing that can go stale and mislead someone later.

Five weeks, and that is fine

Opened 30 June, merged 3 August by **tomaarsen**.

Five weeks for an 18-line documentation PR in a busy repo. Nothing was wrong; it simply was not urgent. I opened the test PR (Day 5) in the same repo during that window, and *that* one merged in two days.

- Do not chase a slow PR with reminders
- Do not open a second PR to "bump" the first
- **Do** keep contributing elsewhere while it sits

The whole workflow, once, end to end

Seven videos in, here is the loop every one of these PRs went through. Nothing here is repo-specific.

Step	Command or action
Fork and clone	gh repo fork owner/repo --clone
Branch	git checkout -b fix-short-description
Install for development	pip install -e "[dev]" (check CONTRIBUTING)
Reproduce	Run the issue's snippet, confirm it fails on main
Fix, then test	Write the test that fails without your change
Run the checks	pytest path/to/test_file.py plus the repo's linter
Push and open	git push -u origin HEAD then gh pr create



The step people skip is "confirm it fails on main". Everything downstream is wasted if the bug is already fixed, and that check costs two minutes.

The one habit that matters most

If you take a single thing from these seven videos, take this: **revert your fix and confirm the test fails.**

- Write the fix and the test
- **git stash only the source change**, leaving the test in place
- Run the test and watch it **fail**
- Restore the fix and watch it **pass**

A test that passes both with and without your change proves nothing, and it is far more common than you would expect. This thirty-second check is the difference between a test and a decoration — and it is the thing a reviewer cannot easily verify for you.

Every merged fix in this series had that check run on it. It is also how I caught one of my own tests asserting my arithmetic instead of the library's actual output.

The seven, honestly

Video	PR	Kind
1	joblib #1812	Real bug — user-reported
2	pypdf #3929	Real bug — maintainer-reported
3	pypdf #3938	Real bug — follow-up, closed a 1-year-old issue
4	nlTK #3703	Tests for untested code
5	sentence-transformers #3855	Tests for untested code
6	joblib #1811	Docstrings
7	sentence-transformers #3843	Docstrings

Three real fixes, two test contributions, two documentation PRs. That is the honest shape of it, and it is the shape I would recommend — but not the order I achieved it in.



What I would do differently

- **Not open thirty-five docs PRs.** Two merged. The same hours spent reading one library's source would have found more bugs.
- **Pick two repos, not twenty.** Every merge above came from a repo where a maintainer had seen my name before.
- **Vet before deep-diving.** I burned days on bugs that were already fixed, or that did not exist in the current version. Check the issue, the linked PRs, and the actual installed source before writing code.
- **Use the library first.** Every real bug here came from code I was already running.

The pattern behind all three real fixes: I was using the library for something else, hit friction, and looked at why. None of them came from browsing "good first issue" labels.

What seven merges are actually worth

Not a job. Let us be accurate about that.

What they are is **evidence** — that you can work inside a codebase you did not write, take review feedback without defending yourself, scope a change, and finish. That is hard to demonstrate any other way, and it is exactly what an interviewer is trying to find out.

The three bug fixes carry that weight. The docs PRs do not, and you should not present them as though they do.

In an interview, do not say "I have seven merged PRs." Say: "A maintainer found a case my fix missed. I reproduced it, asked whether to widen the PR or keep it separate, and we shipped two." That is the story that gets a follow-up question.

Where to start tomorrow

- Pick **one library you actually use** this week
- Open its issue tracker, filter to **bugs filed by maintainers**, sort oldest first
- Find one with **no linked PR**, and reproduce it before writing anything
- If nothing fits — open the test folder and find a **public function with no test**

That is the whole method. Everything in these seven videos came out of it.

One real fix in a library you use beats fifty PRs in repos you have never opened. That took me thirty-five attempts to learn. It should not take you that many.

